

Build & Use a UI Tester Power

In this lab, you'll use the **Build a Power** power to create a custom **UI Tester** power that uses Playwright MCP to automate browser testing. Then you'll vibe code a web app and use your new power to run e2e and smoke tests on it.

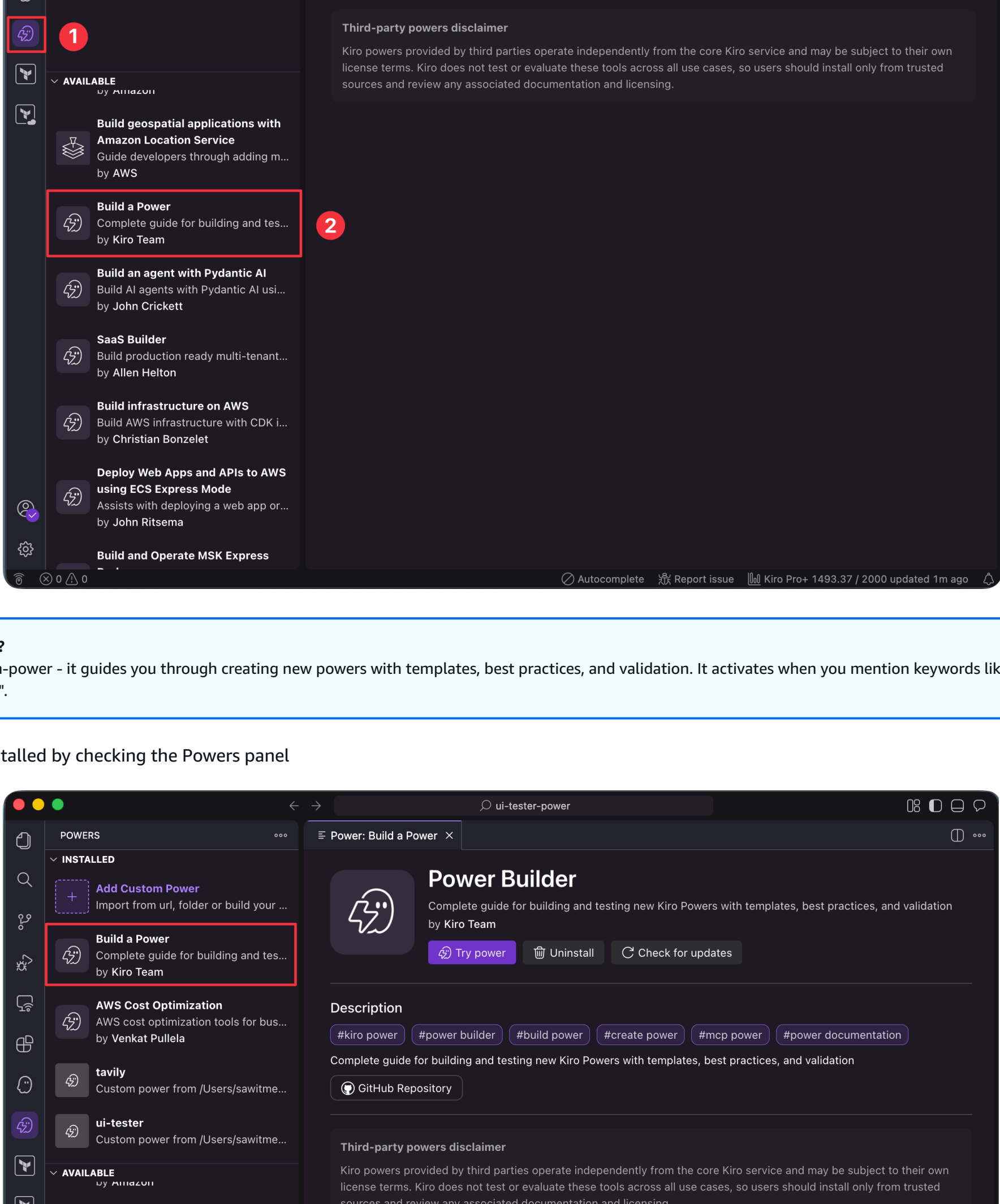
This lab demonstrates the full power lifecycle: create -> install -> use.

What You'll Learn

- How to use the "Build a Power" power to scaffold a new power
- How Powers package MCP tools with steering guidance
- How to install and test a locally-built power
- How to use a power for automated UI testing

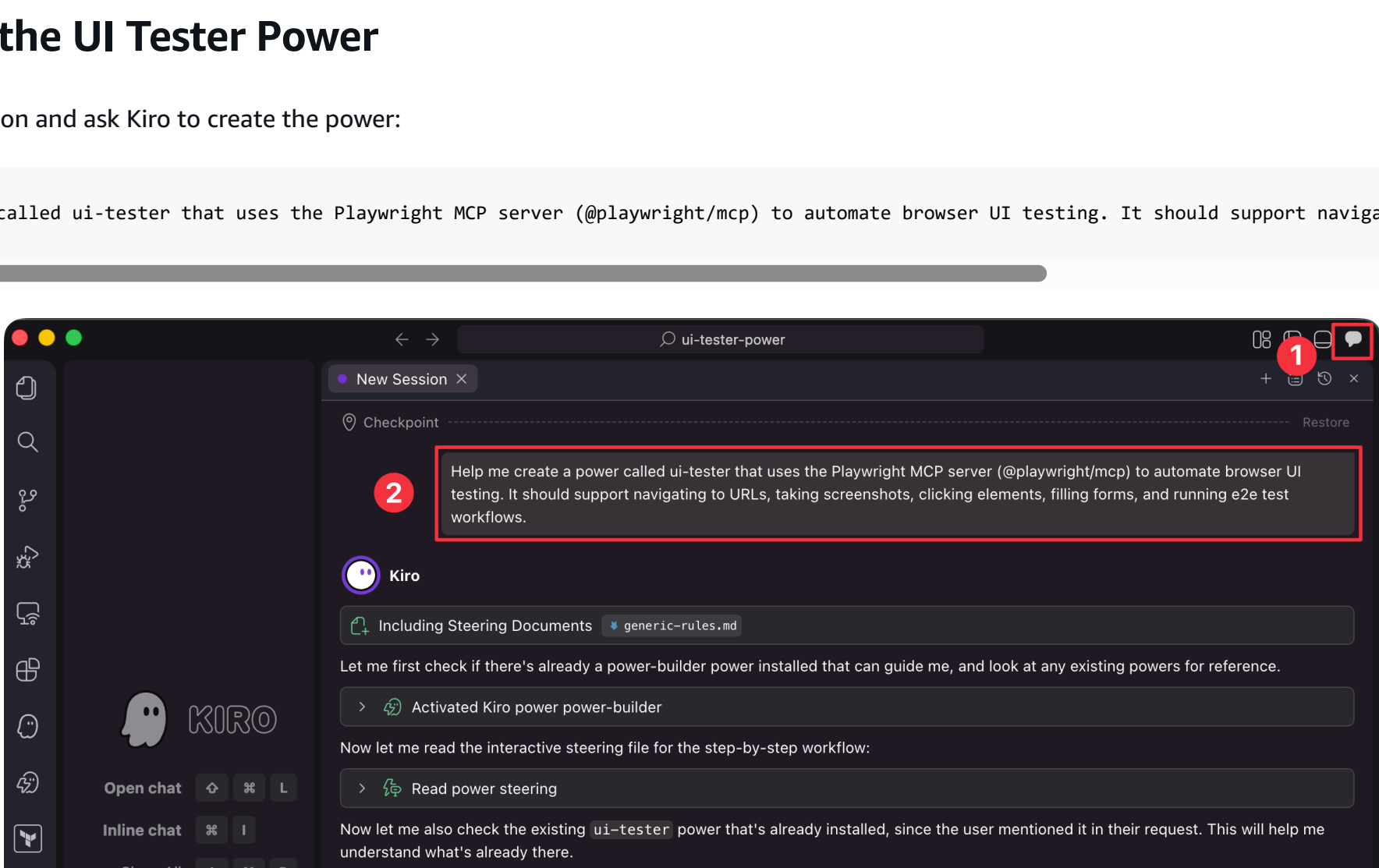
Step 1: Install the "Build a Power" Power

1.1 Find and install **Build a Power** from the marketplace



What is Build a Power?
Build a Power is a meta-power - it guides you through creating new powers with templates, best practices, and validation. It activates when you mention keywords like "build power", "create power", or "new power".

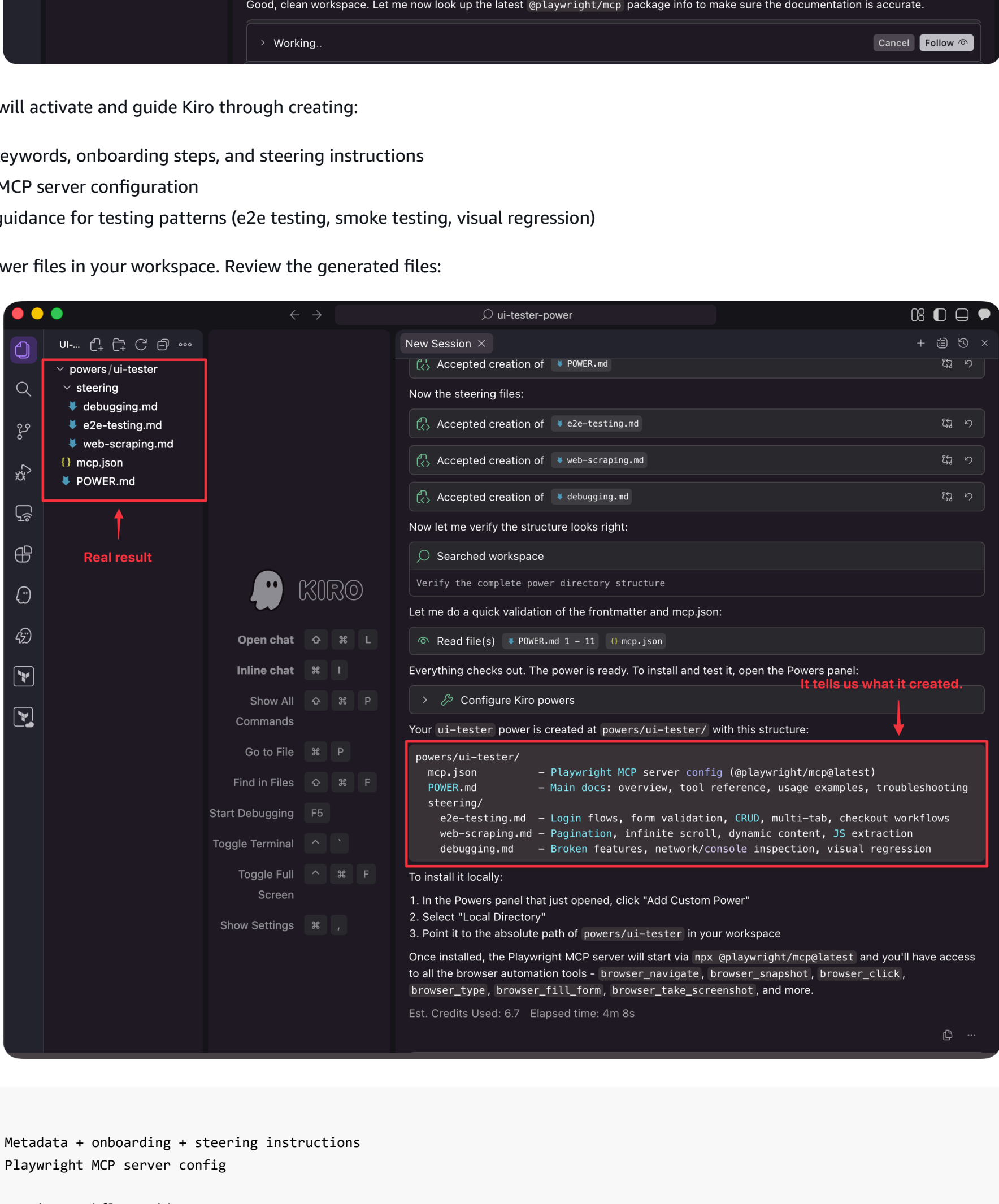
1.2 Verify the power is installed by checking the Powers panel



Step 2: Create the UI Tester Power

2.1 Open a new chat session and ask Kiro to create the power:

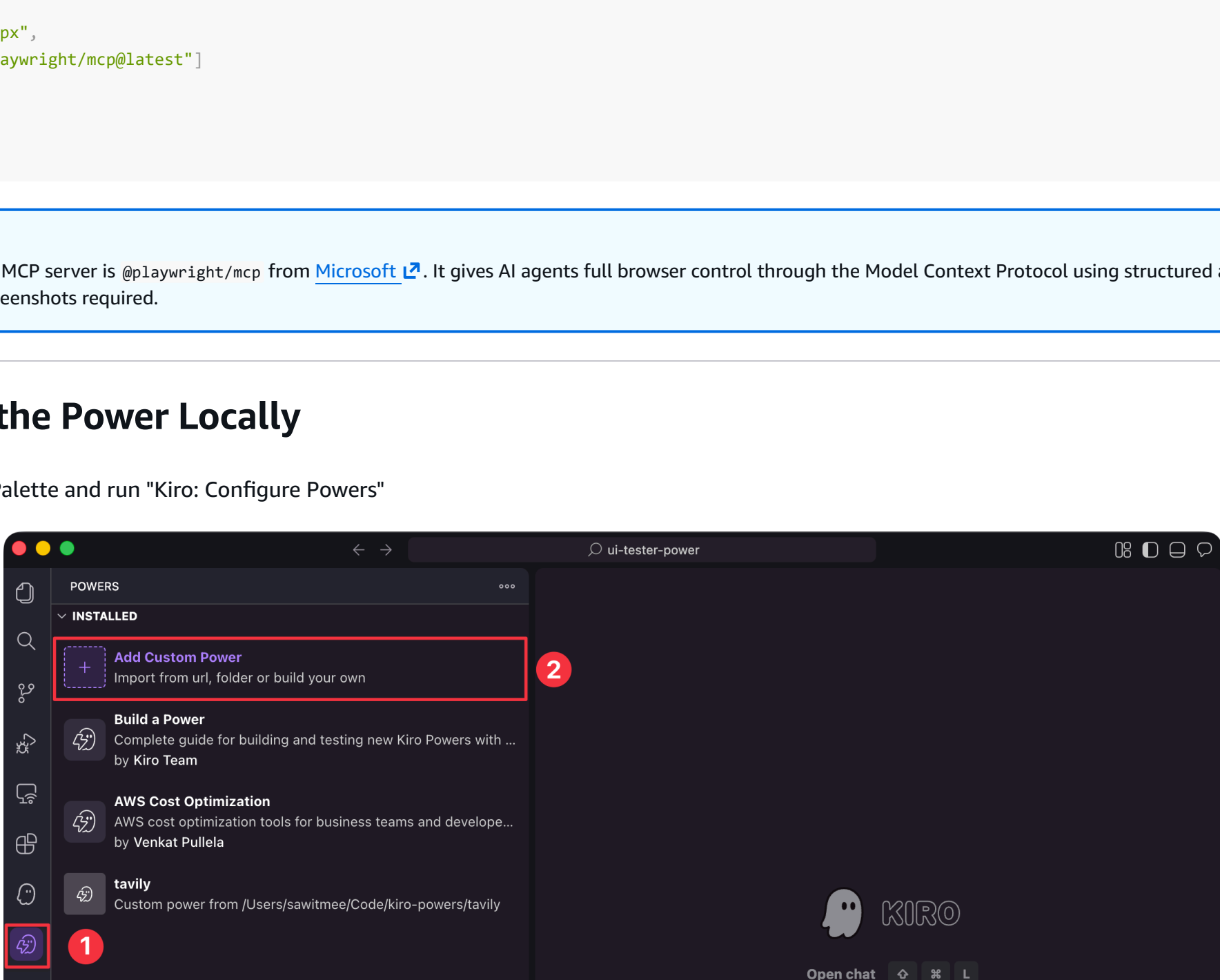
Help me create a power called ui-tester that uses the Playwright MCP server (@playwright/mcp) to automate browser UI testing. It should support navigating to URLs, taking screenshots, clicking elements, filling forms, and running e2e test workflows.



The Build a Power power will activate and guide Kiro through creating:

- power.md - metadata, keywords, onboarding steps, and steering instructions
- mcp.json - Playwright MCP server configuration
- steering/ - workflow guidance for testing patterns (e2e testing, smoke testing, visual regression)

2.2 Kiro will create the power files in your workspace. Review the generated files:



power-ui-tester/
├── power.md
├── mcp.json
├── steering/
└── *.md

2.3 Review the power.md file. It should include:

- Frontmatter with keywords like playwright, browser, testing, ui, e2e, screenshots
- Onboarding section that validates Node.js is installed
- Steering instructions that map testing workflows to steering files

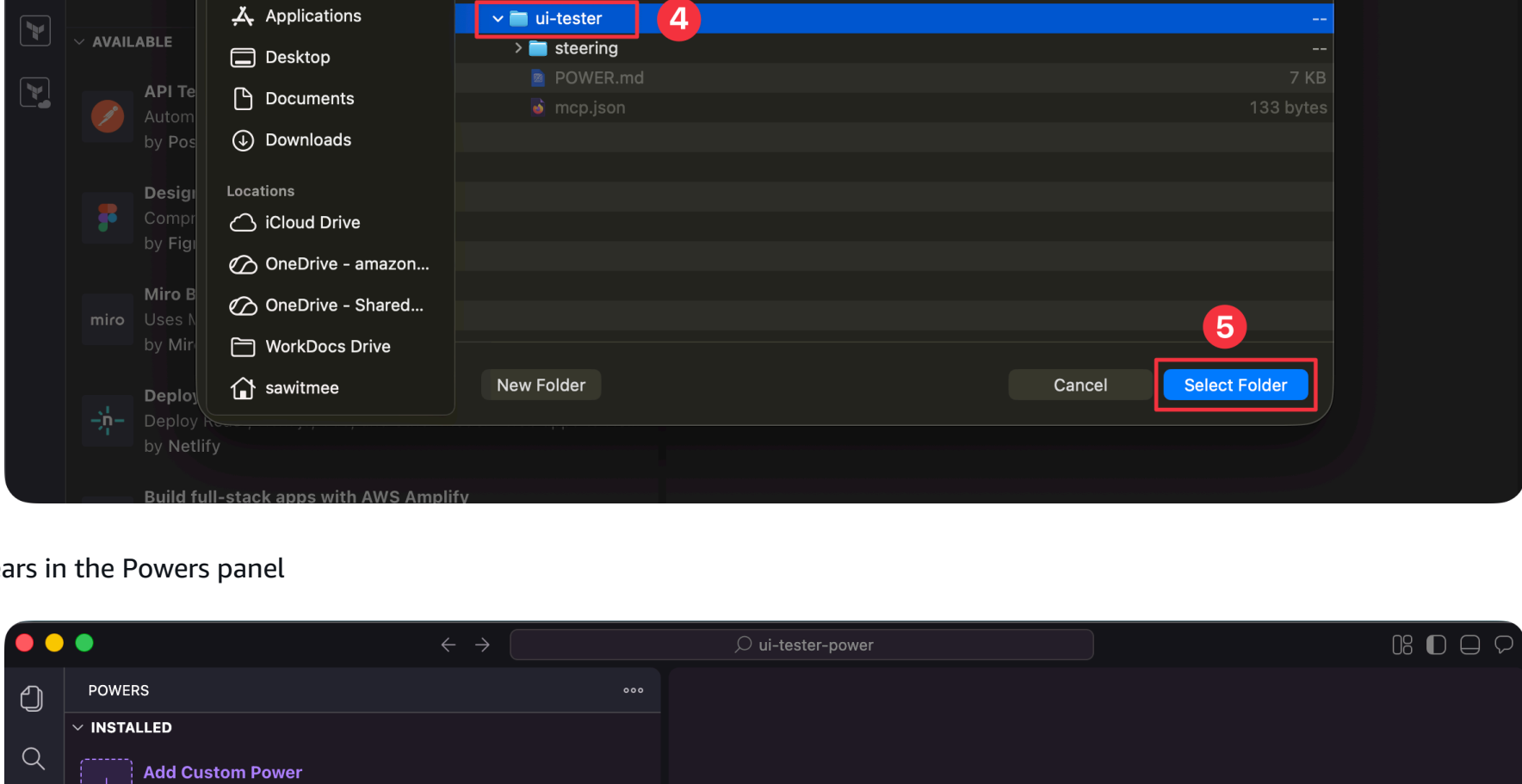
2.4 Review the mcp.json file. It should configure the Playwright MCP server:

```
1 {  
2   "mcpServers": {  
3     "playwright": {  
4       "command": "npx",  
5       "args": ["@playwright/mcp@latest"]  
6     }  
7   }  
8 }
```

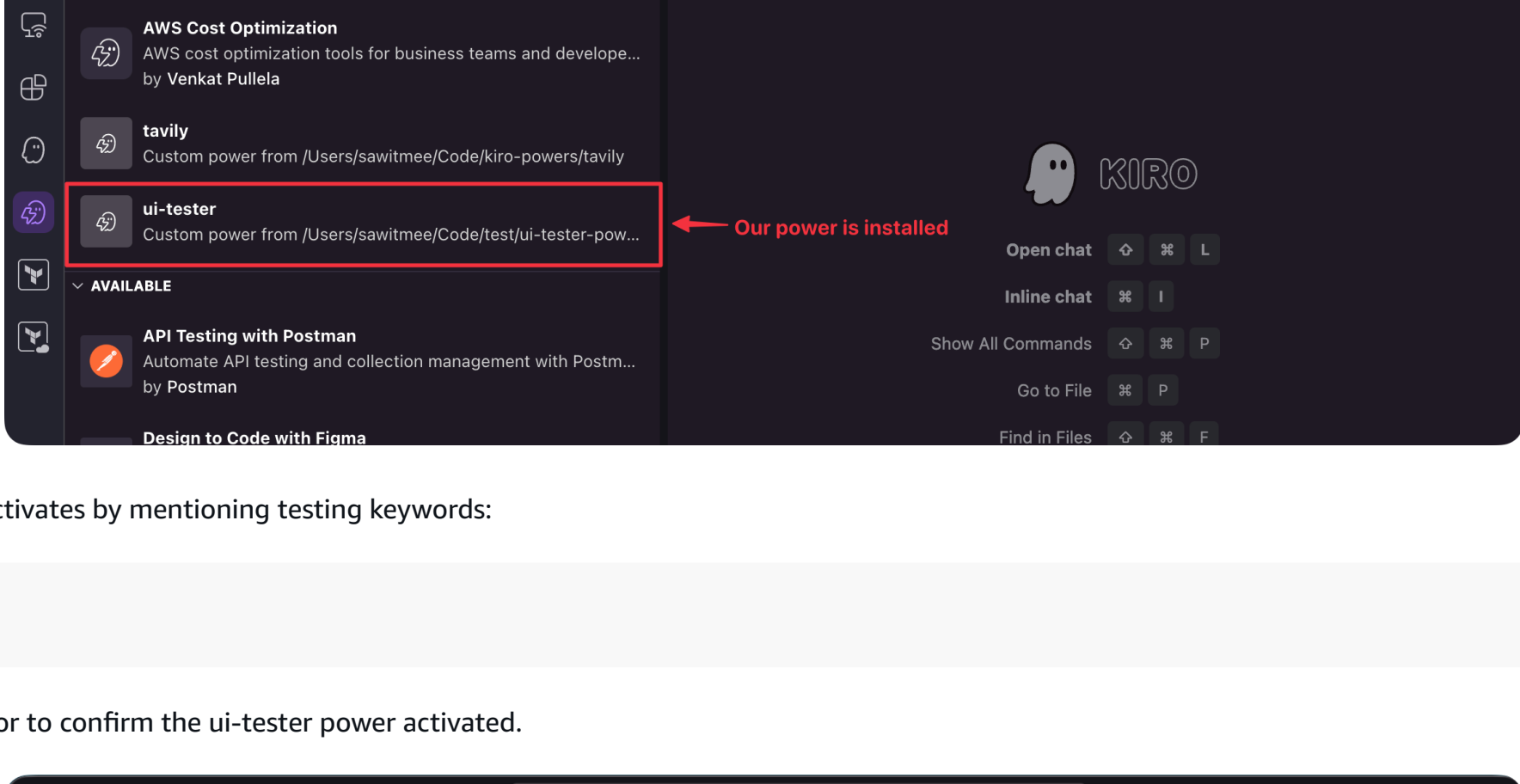
Note
The official Playwright MCP server is @playwright/mcp from [Microsoft](#). It gives AI agents full browser control through the Model Context Protocol using structured accessibility snapshots - no vision models or screenshots required.

Step 3: Install the Power Locally

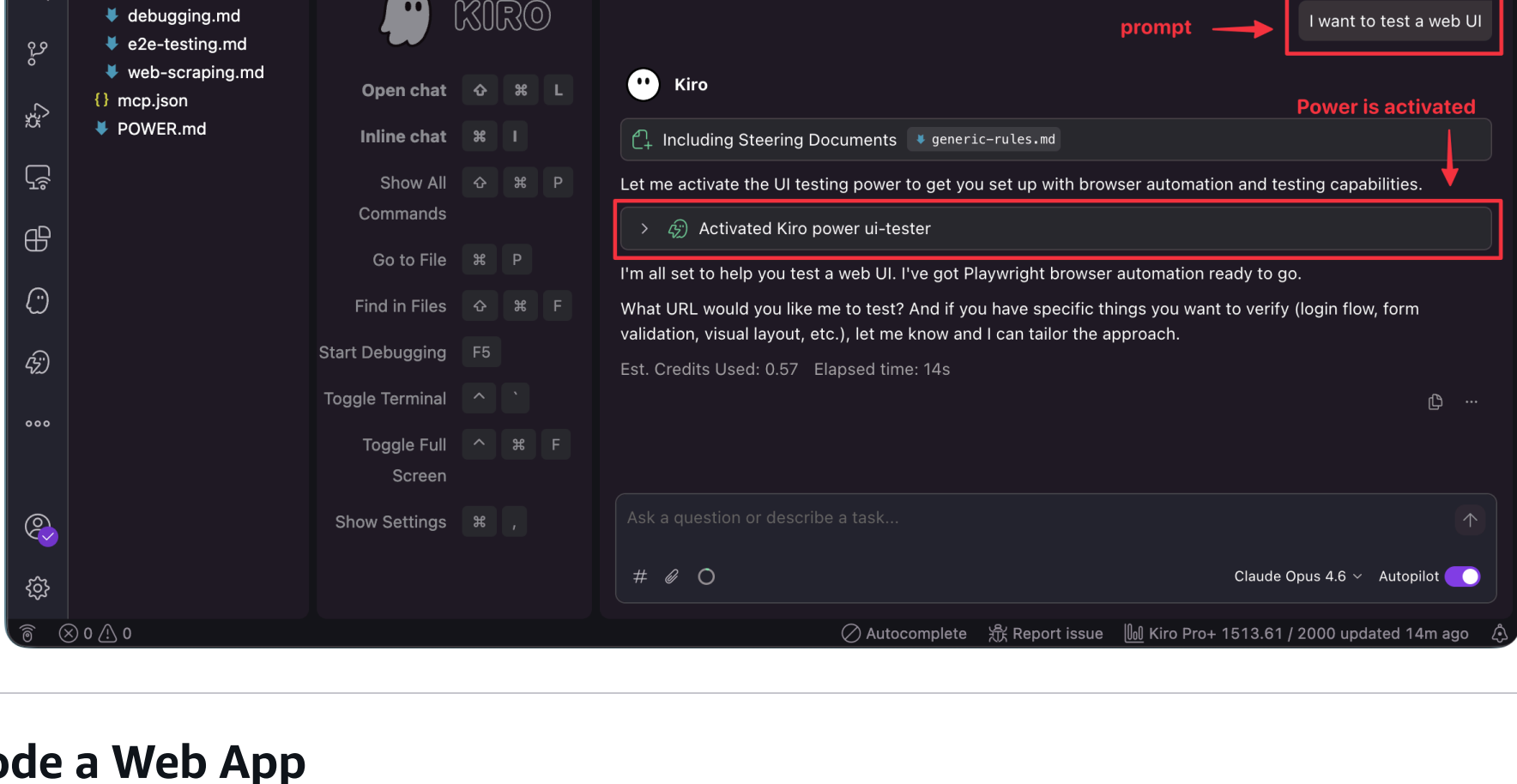
3.1 Open the Command Palette and run "Kiro: Configure Powers"



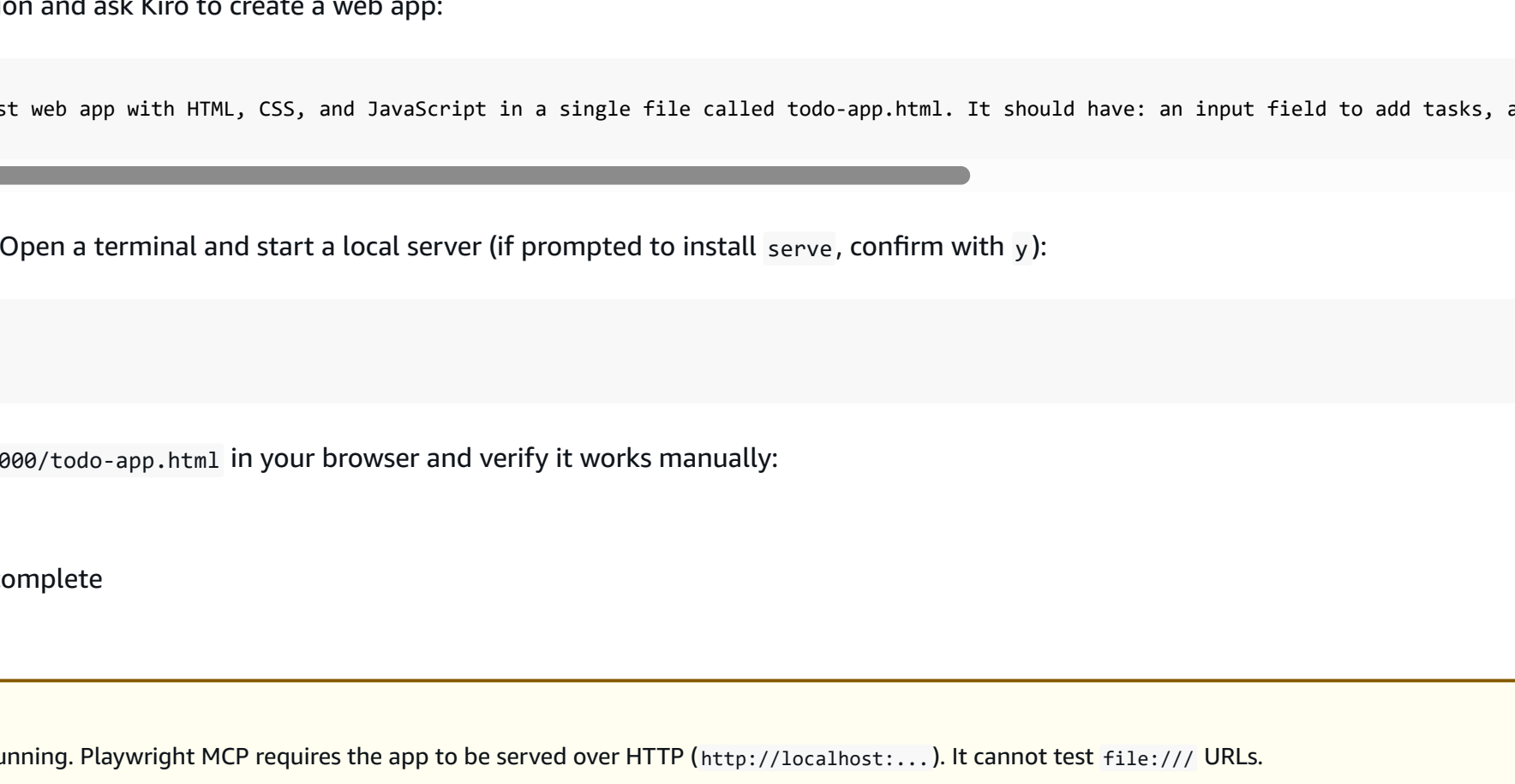
3.2 Click 'Add power from Local Path'



3.3 Navigate to the power-ui-tester/ directory that Kiro created and select it



3.4 Verify the power appears in the Powers panel



3.5 Test that the power activates by mentioning testing keywords:

I want to test a web UI

Check the context indicator to confirm the ui-tester power activated.



Step 4: Vibe Code a Web App

Now create a simple web app to test against.

4.1 Open a new chat session and ask Kiro to create a web app:

Create a simple todo list web app with HTML, CSS, and JavaScript in a single file called todo-app.html. It should have: an input field to add tasks, a button to add them, and a button to delete them.

4.2 Serve the app locally. Open a terminal and start a local server (if prompted to install, serve, confirm with y):

```
1 npx serve -p 8080
```

Open <http://localhost:8080/todo-app.html> in your browser and verify it works manually:

- Add a few tasks
- Click a task to mark it complete
- Delete a task

Important
Keep the local server running. Playwright MCP requires the app to be served over HTTP (<http://localhost:8080>). It cannot test file:/// URLs.

Step 5: Run UI Tests with the Power

Now use your ui-tester power to automate testing of the todo app.

Smoke Test

5.1 Ask Kiro to run a smoke test:

Use the ui-tester power to smoke test <http://localhost:8080/todo-app.html>. Open the page in the browser, verify the page layout, check that the input field and add button are present.

Kiro should:

- Navigate to the todo app
- Take an accessibility snapshot to verify elements are present
- Take a screenshot of the initial state
- Report whether the basic UI elements loaded correctly

E2E Test

5.2 Ask Kiro to run a full e2e test:

Use the ui-tester power to run an e2e test on <http://localhost:8080/todo-app.html>. Test the complete workflow: 1) Add three tasks ("Buy groceries", "Walk the dog", "Read a book"). 2) Mark "Walk the dog" as completed. 3) Delete "Buy groceries". 4) Take a final screenshot.

Kiro should:

- Navigate to the todo app
- Add each task by typing in the input and clicking the add button
- Verify each task appears in the list
- Click "Walk the dog" to mark it complete
- Click the delete button on "Buy groceries"
- Take a screenshot of the final state
- Report a structured pass/fail result for each step

5.3 Review the test results. Kiro should produce a report like:

Step	Action	Expected	Result
1	Add "Buy groceries"	Task appears in list	Pass
2	Add "Walk the dog"	Task appears in list	Pass
3	Add "Read a book"	Task appears in list	Pass
4	Mark "Walk the dog" complete	Task shows as completed	Pass
5	Delete "Buy groceries"	Task removed from list	Pass
6	Final screenshot	Captured	Pass

Key Takeaways

- Build a Power** accelerates power creation by scaffolding the structure and guiding best practices
- Powers package MCP tools (Playwright) with steering guidance (testing workflows) into a reusable bundle
- Once installed, powers activate automatically when you mention relevant keywords
- The same power can be shared with your team via GitHub for one-click installation

- Next Steps**
Try extending your ui-tester power:
 - Add a steering file for visual regression testing (compare screenshots before/after changes)
 - Add a steering file for accessibility testing (check ARIA labels, contrast ratios)
 - Share the power with your team by pushing it to a GitHub repository

Ref: [Create Powers](#) | [Powers Marketplace](#) | [Playwright MCP](#)